

CabinGuard V2 修订稿

修订一：导航地图技术路线

1. 导航平台

导航系统统一改为：

地图展示：高德地图 JS API 2.0
地点输入提示：高德 Autocomplete
地点搜索：高德 PlaceSearch / Web 服务 POI 搜索
地理编码：高德 Geocoding API
驾车路线规划：高德 Driving / Web 服务路径规划 API
路线显示：高德地图折线与路线规划组件
坐标体系：GCJ-02

系统不再使用：

MapLibre
OpenStreetMap
Nominatim
OSRM

这里所说的高德接口是**高德开放平台 API**，需要在高德开放平台创建应用并申请对应的 Key。

2. 前后端职责

Web 前端

使用高德地图 JS API 2.0：

显示当前位置
显示目的地候选点
绘制导航路线
显示实时路况图层
显示起点和终点 Marker
显示预计到达时间和剩余距离
模拟车辆沿路线移动

Python 后端

通过高德 Web 服务 API 完成：

地点名称转经纬度
POI 搜索
候选目的地查询
驾车路线规划
路线距离计算
预计行驶时间计算
导航步骤解析

推荐结构：

用户语音
↓
STT
↓
DeepSeek 判断导航意图
↓
navigation_service
↓
高德地点搜索
↓
候选地点确认
↓
高德驾车路径规划
↓
路线结果返回 Agent
↓
地图更新 + TTS 播报

3. 多轮导航流程

用户：

“带我去虹桥站。”

Agent 调用：

```
navigation_service(  
    action="search",  
    destination="虹桥站"  
)
```

高德返回多个候选地点：

上海虹桥站
虹桥火车站地铁站

虹桥机场 T1
虹桥机场 T2

Agent:

“您是要前往上海虹桥火车站，还是虹桥机场？”

用户:

“火车站。”

系统保留上一轮上下文并调用:

```
navigation_service(  
    action="preview",  
    destination="上海虹桥站"  
)
```

Agent:

“已经找到上海虹桥站，全程约三十二公里，预计四十二分钟。现在开始导航吗？”

用户:

“开始吧，顺便看看那里天气。”

系统连续执行:

```
navigation_service(action="start")  
weather_service(action="destination")
```

最后播报:

“导航已经开始，预计四十二分钟到达。上海虹桥站目前二十三度，有小雨，建议下车时携带雨具。”

地图同步显示:

当前位置
目的地
路线折线
路线距离
预计到达时间
实时路况

4. 两天 Demo 的导航范围

必须完成：

语音输入目的地
地点名称搜索
候选地点确认
驾车路线规划
地图路线绘制
距离与时间显示
目的地天气联动
导航开始语音播报

模拟完成：

车辆沿路线前进
剩余距离变化
下一导航动作播报
到达目的地

暂不完成：

真实 GPS 定位
真实车辆移动
车道级导航
实时导航偏航重规划
真实逐路口连续导航

系统可以提供“模拟前进”按钮，每次点击后推进一段路线，并播放下一条导航指令。

5. 导航工具保持不变

六个高层工具中的导航工具仍然为：

```
navigation_service(  
  action="search | preview | start | cancel",  
  destination="上海虹桥站"  
)
```

但内部实现改为高德开放平台：

search
→ 高德 POI 搜索

preview

→ 高德驾车路径规划

start

→ 保存导航状态并更新地图

cancel

→ 清除当前路线和导航状态

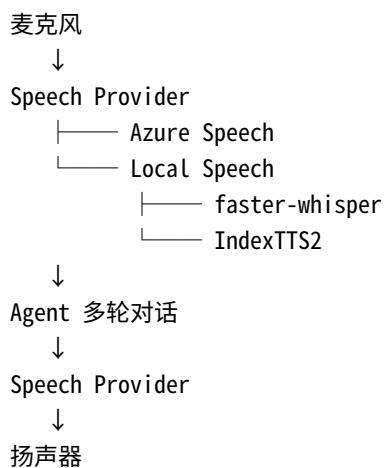
修订二：STT 与 TTS 技术路线

1. 总体原则

语音系统改为：

云端语音 API 优先，本地语音模型作为备用，统一通过可替换的 Provider 接口接入。

系统架构：



2. MVP 默认选择

两天 Demo 默认优先尝试：

STT: Azure Speech to Text
TTS: Azure Text to Speech

主要原因：

STT 与 TTS 使用同一套服务
Python 接入步骤较少
不需要下载和部署大型模型
不占用本地大量显存

中文语音识别和合成可直接测试
容易快速跑通完整闭环
适合单台 PC 和视频演示

实现时优先使用 Azure Speech SDK，而不是手动拼接底层 HTTP 请求：

```
pip install azure-cognitiveservices-speech
```

配置：

```
AZURE_SPEECH_KEY=your_key  
AZURE_SPEECH_REGION=your_region
```

3. STT 交互方式

两天版本仍然使用稳定的按键式交互：

```
点击“开始说话”  
↓  
录制用户语音  
↓  
点击“结束”或检测到静音  
↓  
Azure Speech STT  
↓  
显示识别文字  
↓  
提交给 Agent
```

不在 MVP 中实现：

持续监听
唤醒词
全双工抢话
用户打断 TTS
复杂回声消除

推荐使用半双工流程：

```
用户说话  
→ 系统识别  
→ Agent 处理  
→ 系统播报  
→ 等待下一轮用户输入
```

这不会影响多轮对话。多轮上下文由 LangChain/LangGraph 保存，而不是由 STT 服务保存。

4. TTS 交互方式

Azure TTS 负责生成：

普通对话语音
天气播报
导航播报
座舱状态反馈
安全提醒

建议定义三种语音播报风格：

普通服务

语速：正常
音量：正常
语气：自然、友好

示例：

“已经将座舱温度调到二十三度。”

导航与天气

语速：略慢
数字和道路名称清晰
重要信息前后加入短暂停顿

示例：

“前方路线畅通，预计四十二分钟到达。目的地目前有小雨。”

安全提醒

语速：适中
音量：略高
语气：明确
句子简短

示例：

“检测到严重疲劳风险。请尽快进入休息区。”

可以使用 SSML 控制：

语速
音量
停顿
音高
关键词强调
数字与时间读法

5. 本地备用方案

本地语音方案继续保留：

本地 STT: faster-whisper
本地 TTS: IndexTTS2

使用场景：

Azure Speech 无法申请
Azure 区域网络延迟过高
演示环境无法联网
希望展示本地语音生成能力
希望使用更有个性的中文音色

推荐优先级：

第一阶段：
Azure STT + Azure TTS

第二阶段：
Azure STT + IndexTTS2

第三阶段：
faster-whisper + IndexTTS2

不建议一开始同时部署 faster-whisper 和 IndexTTS2，否则容易将大量时间消耗在：

CUDA 版本
PyTorch 环境
模型权重下载
音频驱动
显存占用
推理延迟

6. Provider 抽象层

无论最终选择 API 还是本地部署，上层 Agent 不应直接依赖某个语音服务。

统一定义：

```
class STTProvider:
    def transcribe(self, audio_path: str) -> str:
        ...

class TTSPProvider:
    def synthesize(
        self,
        text: str,
        style: str = "general"
    ) -> str:
        ...
```

实现：

```
AzureSTTProvider
AzureTTSPProvider
FasterWhisperSTTProvider
IndexTTS2Provider
```

配置文件：

```
STT_PROVIDER=azure
TTS_PROVIDER=azure

STT_FALLBACK=faster_whisper
TTS_FALLBACK=index_tts2
```

调用关系：

```
text = stt_provider.transcribe(audio_path)

agent_response = cabin_agent.invoke(text)

audio_path = tts_provider.synthesize(
    agent_response,
    style="navigation"
)
```

这样后续切换语音方案不需要修改 Agent、天气、导航和安全门控代码。

7. 降级与容错

语音服务失败时：

Azure STT 超时
→ 尝试本地 faster-whisper
→ 再失败则允许键盘输入

Azure TTS 超时
→ 尝试 IndexTTS2
→ 再失败则只显示文字

每次调用设置超时：

STT：建议 8~12 秒
TTS：建议 8~15 秒

页面必须始终显示文字，因此即使语音接口失败，Agent 主流程也能继续演示。

8. 视频展示稳定性

录制正式视频前：

固定三个主要演示场景
测试 Azure 网络延迟
缓存固定的安全提醒音频
缓存天气播报模板
缓存导航开始和到达提示
准备键盘输入入口
保留本地 TTS 或系统 TTS 备用

需要缓存的固定音频包括：

导航已开始
已经到达目的地
检测到疲劳驾驶风险
行驶过程中不能播放视频
建议尽快进入休息区
天气查询暂时不可用

动态内容如温度、目的地和预计时间仍可以实时生成。

修订三：Web 页面中的地图与语音

中间区域调整为高德地图：

高德地图

当前位置 ●	路线	★ 目的地
32.4 km	预计 42 分钟	
当前路况：轻度拥堵		

底部语音区域：

[开始录音] [停止录音] [重新识别] [停止播报]

当前语音服务：

STT: Azure Speech

TTS: Azure Speech

当前状态：

正在录音

正在识别

正在查询高德地图

Agent 正在处理

正在生成语音

正在播报

设置区域允许切换：

STT Provider:

☐ Azure Speech

☐ faster-whisper

TTS Provider:

☐ Azure Speech

☐ IndexTTS2

修订四：项目目录

```

cabin-guard-v2/
├── app.py
├── backend/
│   ├── api.py
│   └── websocket.py
├── agent/
│   ├── cabin_agent.py
│   ├── prompts.py
│   ├── conversation_state.py
│   └── schemas.py
└── speech/

```

```
|   |—— interfaces.py
|   |—— recorder.py
|   |—— audio_queue.py
|   |—— azure_stt.py
|   |—— azure_tts.py
|   |—— faster_whisper_stt.py
|   |—— index_tts2_provider.py
|   |—— speech_manager.py
|—— mcp_servers/
|   |—— weather/
|       |—— server.py
|       |—— weather_client.py
|—— navigation/
|   |—— amap_client.py
|   |—— poi_search.py
|   |—— geocoder.py
|   |—— route_service.py
|   |—— navigation_state.py
|—— tools/
|   |—— weather_tool.py
|   |—— navigation_tool.py
|   |—— climate_tool.py
|   |—— media_tool.py
|   |—— seat_tool.py
|   |—— safety_tool.py
|—— frontend/
|   |—— dashboard.py
|   |—— amap_component/
|       |—— index.html
|       |—— map.js
|       |—— map.css
|—— ...
```

修订五：两天开发顺序

第一天上午

连接 DeepSeek V4
建立 Agent 多轮上下文
完成六个高层工具接口
完成 Safety Gate

第一天下午

申请并测试 Azure Speech
跑通麦克风 STT
跑通 Azure TTS

完成 Speech Provider 抽象
跑通语音多轮对话

第一天晚上

申请高德开放平台 Key
在 Web 页面加载高德地图
完成 POI 搜索
完成驾车路线规划
绘制路线

第二天上午

完成天气 MCP
完成目的地天气联动
完成高德导航与语音播报联动
完成导航多轮确认

第二天下午

补充座椅控制
补充驾驶员状态场景
接入 LangSmith
加入语音和地图降级方案
固化三个演示场景
录制视频

如果 Azure Speech 在第一天下午无法稳定工作，则立即切换：

STT → faster-whisper
TTS → IndexTTS2 或系统 TTS

不要等到第二天再决定语音技术路线。

修订后的默认技术选型

LLM：
DeepSeek V4 API

Agent：
LangChain / LangGraph

天气：
自建天气 MCP

地图与导航：

高德地图 JS API 2.0

高德 Web 服务 API

STT：

Azure Speech 优先

faster-whisper 备用

TTS：

Azure Speech 优先

IndexTTS2 备用

Web：

Streamlit + FastAPI + 高德地图组件

追踪：

LangSmith

记忆：

SQLite / JSON

EverOS 可选

安全：

独立 Safety Gate

除以上导航与语音技术路线调整外，CabinGuard V2 的项目定位、主动服务、天气 MCP、多轮交互、六个高层工具、座椅控制、驾驶员状态、安全门控、记忆方案、LangSmith、Web 页面和视频演示方案均保持不变。